

Agentic AI Project Operating Plan for a Rubric-Driven School ML Project

Executive summary

The project should be run as a **rubric-driven, evidence-producing engineering program**, not as an open-ended attempt to add more AI features. Based on the provided assignment materials, the immediate objective is unusually clear: the workbook rewards breadth and rigor of model/system design, while Demo #1 rewards **working evidence** of preprocessing, training/test preparation, analytics, early machine-learning results, and a coherent presentation. The demo is due **October 6, 2026 at 12:00 PM**, giving roughly two weeks from the current date. [filecite\turn0file0](#) [filecite\turn0file1](#)

There is already a substantial project foundation. The previous workbook and presentation describe a Jason Lemkin/SaaStr persona-generation system with approximately 31,774 raw records, PySpark preprocessing, DuckDB, ChromaDB, MinIO, Airflow, Ollama, Streamlit, PersonaRAG, and QLoRA. [filecite\turn0file2](#) [filecite\turn0file3](#) The problem is **not lack of sophistication**. The bigger risk is that the project has accumulated multiple inconsistent versions of its own architecture, datasets, model definitions, and training configuration.

The most important findings are:

Finding	Why it matters	Recommended action
The workbook rubric's highest model-proposal rating requires five or more models and at least one innovative/improved model, while the previous project narrative centers on four models. filecite\turn0file0 filecite\turn0file2	A strong four-model implementation can still miss the explicit "Excellent" criterion.	Define a fifth model now, preferably Hybrid PersonaRAG + QLoRA , but clearly distinguish <i>proposed</i> from <i>implemented</i> .
The public default GitHub view returned in this research pass is still a Paul Graham version on <code>master</code> , with files such as <code>pipeline.ipynb</code> , <code>cleaned_pg.csv</code> , <code>persona_profile_pg.json</code> , and <code>train_pg.csv</code> , whereas the previous workbook says <code>etl_jason1</code> is the current Jason Lemkin production branch. ¹ filecite\turn0file3	An agent pointed at the default branch can optimize the wrong project. It also damages reproducibility and grader confidence.	Make the production branch the source of truth, merge it into a clean protected main branch, or at minimum document the canonical branch/commit explicitly before any agent edits code.

Finding	Why it matters	Recommended action
The previous slides themselves contain architectural drift: one slide labels the four approaches Hybrid RAG/CharLoRA/Persona-Plug/RAFT+APR, while the next describes Base Llama/Prompt+Persona/PersonaRAG/QLoRA. Other inconsistencies include Chroma collection names, DAG/task counts, evaluator filenames, and QLoRA target modules/hardware. filecite:turn0file2 filecite:turn0file3	Results can become irreproducible and slides can contradict code during the demo.	Create one machine-readable <code>configs/project.yaml</code> plus an architecture/status manifest and generate tables/diagrams from it.
The previous workbook cites a 169/21/22-document train/validation/test split totaling 212 documents while discussing a current corpus of roughly 5,800 processed records, and explicitly says the split strategy originated in the older <code>scrapers</code> branch. filecite:turn0file3	This strongly suggests the split manifest may be a legacy artifact. If so, evaluation leakage or simply evaluating the wrong corpus is a greater threat than model quality.	Recompute and freeze splits on the current Jason corpus before running final experiments. Group duplicates/cross-posts before splitting.
The demo rubric allocates equal weight to preprocessing, training/test data, analytics, ongoing ML results, and presentation. filecite:turn0file1	Spending the remaining time only on QLoRA or a fifth architecture is strategically wrong.	Secure full-credit evidence for all five demo categories first; treat expensive fine-tuning and advanced retrieval as stretch work.

The recommended operating stack is deliberately conservative: **Cursor for interactive repository work; GitHub branches, pull requests and CI as the control plane; `uv` for reproducible Python environments; Ruff + pytest for fast quality gates; DVC or equivalent manifests for data/artifact versioning; and the existing Airflow/DuckDB/Chroma/MinIO/Ollama stack rather than adding a new orchestration framework.** `uv`'s lockfile records exact resolved dependencies and is intended to be committed for consistent installs; DVC supports Git-coupled data tracking, reproducible pipelines and experiments. ²

For AI-assisted development, the core principle should be:

Agents may accelerate implementation and analysis; humans own scope, datasets, evaluation definitions, experimental conclusions, merges, and final claims.

Cursor explicitly provides read-only Ask mode as well as autonomous Agent workflows; its current local-agent execution modes add sandboxing/review controls. Background agents, by contrast, run on isolated remote machines with internet access and automatically execute terminal commands, and Cursor warns that this creates prompt-injection and data-exfiltration risk. ³ Thus, research PDFs and web content should be treated as **untrusted data**, not as instructions an autonomous coding agent is allowed to execute.

The deadline strategy should be to have **data and baselines stable by September 27, comparative results by September 30–October 1, a hard results/configuration freeze on October 2, recorded demo material by October 4, and October 5 reserved for rehearsal and upload**. QLoRA or the fifth model should never be allowed to jeopardize these milestones.

Evidence ingestion and project knowledge base

The project should begin with a formal ingestion pass that turns every input—rubrics, old workbook, old slides, repository, research PDFs, notebooks, and training data—into a traceable project knowledge base. This is one of the highest-leverage uses of agentic AI because it is primarily a **read, compare, classify, and cross-reference** task rather than a risky code-writing task.

What has already been established

The provided workbook rubric requires detailed model proposals, support environments and diagrams, model comparison and justification, model-specific evaluation, system boundaries/use cases, architecture and cloud/platform integration, data/database design, UI/visualization, implementation details, APIs, and the supporting environment. Most importantly, the “Excellent” model-proposal band asks for at least five models and at least one innovative or improved model. [filecite turn0file0](#)

The Demo #1 rubric requires evidence in five equally weighted areas: preprocessing, train/test preparation, analytics, ongoing ML results, and presentation. It additionally requires presentation slides and a video covering those items and current platform status. [filecite turn0file1](#)

The old artifacts are extremely useful but should be treated as **historical evidence, not source-of-truth configuration**. For example, the presentation claims approximately 31,774 raw records from five sources and a path through PySpark, DuckDB, ChromaDB, MinIO and four model variants, while the prior workbook narrates the migration from a Paul Graham prototype to `etl_jason1`. [filecite turn0file2](#)
[filecite turn0file3](#)

Recommended ingestion architecture

Create these directories before asking an agent to “understand the project”:

```
project/
├── docs/
│   ├── rubrics/
│   ├── prior_work/
│   ├── architecture/
│   ├── research/
│   └── rubric_matrix.md
├── research/
│   ├── raw/           # immutable source PDFs, exported slides, papers
│   ├── extracted/     # page-preserving text/markdown
│   ├── index/         # optional research-only semantic index
│   └── research_manifest.csv
```

```

├─ data/
│   ├─ raw/           # immutable collected data
│   ├─ interim/
│   ├─ processed/
│   └─ manifests/
├─ configs/
├─ src/
├─ tests/
├─ notebooks/
├─ experiments/
├─ reports/
│   ├─ figures/
│   ├─ tables/
│   └─ demo1/
└─ artifacts/

```

The essential design decision is to keep **research literature separate from persona/training data**. Research PDFs belong in a research index. Jason Lemkin content belongs in the modeling corpus. Accidentally placing academic papers in the same retrieval collection as persona content would contaminate retrieval behavior and invalidate claims about persona grounding.

For Google Drive, use a read-only ingestion process. Drive API v3 supports listing/searching files, retrieving file IDs, downloading content, and exporting Google Workspace documents; Google explicitly provides a `drive.readonly` authorization scope. ⁴ Native Google Workspace documents should be exported to stable project formats—for example Slides to PPTX/PDF—while existing PDFs and data blobs should be downloaded without modification. The raw copies should then receive cryptographic hashes.

A practical `research_manifest.csv` should contain:

```

artifact_id
source_type
title
original_location
local_path
mime_type
modified_at
sha256
authors
publication_date
tags
rubric_relevance
project_relevance
citation
ingestion_status
notes

```

For PDFs, extraction should retain **page boundaries** so an agent cannot turn a page-specific source into an unverifiable floating quotation. Slides require both textual extraction and visual inspection because architecture diagrams and charts often encode information that is absent from plain-text extraction. That is already true of the old project presentation, where some of the most important inconsistencies are apparent in the model-comparison slides. [filecite00turn0file200](#)

Rubric-first ingestion

The first AI-generated artifact should not be a code plan. It should be a **rubric traceability matrix**:

Rubric criterion	Exact requirement	Current evidence	Missing evidence	Artifact to produce	Owner	Status
Model proposals	≥5 for Excellent; detailed architecture/ features/ algorithms	Four canonical models documented	Fifth improved model	Model comparison + M5 specification	Model lead	Gap
Model supports	Platform/ tools + strong diagrams	Prior infra diagram	Canonical/ current diagram	Architecture diagram generated from config	Platform lead	Partial
Model evaluation	Method/ metrics per model	BERTScore, RAGAS, style, BLEU/ ROUGE, G-Eval described	Retrieval metrics, frozen protocol	Experiment protocol + results	Evaluation lead	Partial
Demo preprocessing	Complete/ clear preprocessing evidence	Pipeline documented	Re-run evidence from current dataset	Before/after + attrition plots	Data lead	Required
Demo train/ test	Complete train/test preparation	Historical split described	Current split/ leakage verification	Split manifest + QA report	Data lead	Critical
Demo ML results	High-quality ongoing results	Framework/ status claims	Reproducible current numbers	Results table + sample outputs	Model lead	Critical

The matrix should become the project backlog. This prevents a coding agent from optimizing locally interesting tasks that are irrelevant to the grade.

Research-agent prompt

A good first Cursor Ask/Plan prompt is:

```
ROLE: Evidence and rubric analyst. Read only; do not edit code or run destructive commands.

INPUTS:
- all rubric files
- prior workbook and presentation
- research_manifest.csv
- repository documentation/configuration

TASK:
1. Extract every explicit grading requirement.
2. Map each requirement to existing evidence and exact artifact locations.
3. Identify contradictions between old documents, configuration, and current code.
4. Separate VERIFIED, CLAIMED-IN-DOCUMENT, INFERRED, and MISSING facts.
5. Produce docs/rubric_matrix.md and docs/project_inconsistencies.md.
6. For every numerical project claim, identify its provenance.
7. Do not invent results and do not resolve contradictions silently.

EXIT CONDITION:
A human can see exactly what must exist for each rubric item and which claims still need verification.
```

This is a good fit for Cursor's read-only exploration mode; Cursor documents Ask mode specifically for understanding a codebase without edits, and Plan mode can be used before implementation. ⁵

Codebase mapping and data architecture

The repository should be mapped **before** any broad “improve this project” prompt. Agentic coding tools become dramatically less reliable when several historical architectures coexist and no explicit source of truth exists.

Preliminary repository diagnosis

The public default GitHub view inspected for this report shows `master` and only the older Paul Graham-oriented root artifacts: `pipeline.ipynb`, `build_index.py`, `generate.py`, `persona_profile_pg.json`, `raw_pg.csv`, `cleaned_pg.csv`, train/validation/test CSVs, and `user_interface.py`. Its repository description likewise identifies it as a Paul Graham-style RAG/Ollama/Streamlit project. ¹

The previous workbook says the project subsequently evolved through three stages:

1. `scrapers` — Paul Graham notebook prototype.
2. `etlpipeline` — Docker/Airflow/Spark/MinIO/Chroma/Snowflake.
3. `etl_jasonl` — current Jason Lemkin implementation with DuckDB replacing Snowflake.

That makes **branch normalization the first engineering task**, not a cosmetic Git cleanup. Any Cursor/Codex agent launched from default `master` could otherwise work against the wrong persona, dataset and architecture.

The correct sequence when the full repository is made available is:

```
git status
git branch -a
git log --graph --decorate --oneline --all
git ls-files
git diff master...<candidate-production-branch> --stat

# Environment/dependency map
uv tree

# Test inventory
pytest --collect-only -q

# Docker/service topology
docker compose config

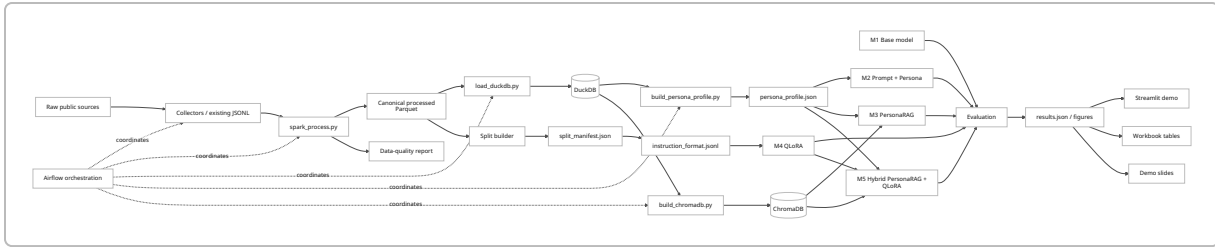
# Airflow/DAG discovery
rg "DAG\\(|@dag|task_id|BashOperator|PythonOperator" .

# Python import/entry-point inventory
rg "^\\s*(from|import) " src dags spark_jobs .
```

The cartography agent should then generate `docs/codebase_map.md`, an entry-point table, a module/import graph, Docker service graph, Airflow DAG graph, and test coverage inventory.

Canonical dependency graph

The following graph is the **recommended normalized architecture**, derived from the prior project artifacts but intentionally resolving the architectural ambiguities rather than pretending every old diagram is simultaneously correct. [filecite:turn0file2](#) [filecite:turn0file3](#)



That graph also provides the natural test boundaries.

Module and test map

Component	Key responsibility	Highest-value tests	CI or local?
Collectors	Acquire/provenance raw content	Schema fixture; missing field handling; source ID stability	Local/ integration
Cleaning	Deterministic normalization	HTML decode, URL removal, markdown flattening, whitespace, empty text	CI
Deduplication	Prevent repeated/cross-platform samples	Exact duplicates; normalized duplicates; duplicate-group IDs	CI
Feature extraction	Stylometric features	Known sentence lengths/TTR/questions/B2B flags	CI
Split builder	Train/val/test isolation	No ID overlap, no duplicate-group overlap, deterministic seed	CI blocker
DuckDB loader	Structured analytics	Table counts, schema, null constraints, tiny fixture pipeline	CI
Chunker	Retrieval units	Chunk boundaries, overlap, source metadata, deterministic IDs	CI
Chroma/retrieval	Candidate retrieval	Index populated, stable metadata, top-k fixture expectations	Integration
Persona builder	Profile generation	Determinism, expected fields, no test-set input	CI
Prompt/model adapters	Common model interface	Prompt construction, no accidental context leakage	CI
Evaluation	Metrics and output schema	Known fixtures, edge cases, no test reference used at generation time	CI blocker
Streamlit/API	User-facing demo	Startup/smoke test, graceful missing-model behavior	Local/ integration

The visible public default root does not show a `tests/` directory or GitHub workflow directory among the listed files, so they should not be assumed to exist in the canonical branch until verified. ¹

Data formats and versioning

The current project already has a reasonable multi-format strategy: JSONL at ingestion/training boundaries, Parquet for processed analytical data, DuckDB for structured querying, and ChromaDB as a derived vector index. The previous materials describe roughly 31,774 raw source records being reduced to roughly 6,200 after the 50-word rule and approximately 5,800 after deduplication, with training records exported in instruction JSONL form.

The missing piece is **artifact identity**.

Every data build should emit something like:

```
{
  "dataset_version": "lemkin-v2.1",
  "created_at_utc": "2026-09-23T04:15:00Z",
  "git_sha": "<commit>",
  "schema_version": "2",
  "raw_source_hashes": {
    "blog": "<sha256>",
    "twitter": "<sha256>",
    "linkedin": "<sha256>",
    "lemkin_youtube": "<sha256>",
    "saastr_youtube": "<sha256>"
  },
  "raw_count": 0,
  "processed_count": 0,
  "duplicate_groups": 0,
  "split_manifest": "splits-v2.json",
  "split_seed": 42,
  "pipeline_config_hash": "<sha256>"
}
```

The zeros above are intentionally placeholders: the manifest must be generated by the pipeline, not copied from slides.

Recommended storage policy:

Artifact	Canonical form	Git?	Data/artifact versioning?
Source code/config	<code>.py</code> , <code>.yaml</code> , <code>.toml</code>	Yes	Git
Raw scraped/downloaded data	JSONL/original	No	DVC/object storage

Artifact	Canonical form	Git?	Data/artifact versioning?
Canonical processed corpus	Parquet	No	DVC/object storage
Split assignment	JSON/Parquet	Yes if small	Git + dataset hash
Training examples	JSONL	Usually no	DVC
DuckDB database	Rebuildable derived artifact	Usually no	DVC/cache
Chroma index	Rebuildable derived artifact	No	DVC/cache
LoRA adapters/GGUF	Model artifact	No	DVC/model storage
Experiment metrics	JSON/CSV/Parquet	Yes	Git
Figures/tables used in reports	PNG/SVG/CSV	Yes	Git
Research PDFs	Immutable research store	Usually no	manifest + object storage

DVC's documented workflow couples `dvc.yaml` pipelines, tracked data and `dvc repro` / `dvc exp` with Git while keeping large data in remote storage. ⁶ For a deadline-constrained student project, that is more useful than introducing a full enterprise MLOps platform.

The **split manifest deserves special treatment**. The previous workbook's 212-document split is suspect because it is explicitly inherited from the old branch while the current corpus is substantially larger. That is an inference, not proof, but it is important enough to stop final evaluation until verified.

filecite turn0file3 The regenerated split should occur at a document/duplicate-group level *before* chunking; all chunks from one original/cross-post group must inherit the same split.

Experiment design and reproducibility

The model program should serve two different goals:

- The **workbook** needs enough conceptual breadth and justification to meet its five-model criterion.
- The **demo** needs credible evidence that some models actually run.

Those goals should not be conflated.

Canonical five-model portfolio

Model	Mechanism	Purpose	Demo priority	Workbook status
M1 Base Llama	Same base model, no persona/retrieval/tuning	Establish the counterfactual baseline	Must run	Implemented/validate
M2 Prompt + Persona	Structured persona profile in prompt	Cheap, interpretable persona conditioning	Must run	Implemented/validate
M3 PersonaRAG	Persona-aware query/retrieval/reranking + generation	Add evidence grounding	Must run if retrieval operational	Implemented/validate
M4 QLoRA	Parameter-efficient tuning over persona examples	Learn stylistic behavior in adapters	High-value but compute-dependent	Implement only if ready
M5 Hybrid PersonaRAG + QLoRA	Fine-tuned model plus retrieval and grounding controls	Test whether style learning and external grounding are complementary	Stretch for demo	Propose for workbook even if not yet trained

RAG's foundational architecture combines a model's parametric knowledge with retrieved non-parametric memory; that is the correct conceptual justification for separating M3 from prompt-only M2. ⁷ PersonaRAG specifically explores personalization of the retrieval/generation process, making it relevant to the project's persona-conditioned retrieval hypothesis. ⁸ QLoRA trains low-rank adapters through a frozen 4-bit-quantized model and was developed explicitly to reduce fine-tuning memory requirements. ⁹

M5 is particularly useful because it repairs the rubric gap without inventing a completely unrelated fifth architecture. It asks a clean scientific question:

Does combining retrieval-grounded evidence with a style-adapted generator improve factual faithfulness without reducing persona fidelity?

That is more defensible than adding another model solely to satisfy a count.

Fix configuration drift before experimenting

The old workbook and subsequent presentation disagree about the QLoRA implementation. The workbook describes four attention target modules, nonzero dropout and a Colab A100-oriented run; the later slides describe seven target modules including MLP projections, zero dropout and a local RTX 5070 Ti run. The

evaluator is also called `persona_pipeline/evaluation/evaluate.py` in the workbook and `eval_rag.py` in the slides. [filecite:turn0file2](#) [filecite:turn0file3](#)

That should become a single experiment configuration:

```
experiment:
  id: m4_qlora_v3

dataset:
  version: lemkin-v2
  split_manifest: data/manifests/splits-v2.json

model:
  base_model: "<exact model revision>"
  method: qlora

lora:
  rank: 16
  alpha: 32
  dropout: "<chosen value>"
  target_modules:
    - "<explicit modules>"

training:
  epochs: "<value>"
  learning_rate: "<value>"
  max_seq_length: "<value>"
  seed: 42

generation:
  temperature: "<value>"
  top_p: "<value>"

evaluation:
  prompt_set: eval-prompts-v2
```

The report should never state one configuration while the code executes another.

Evaluation matrix

The current project already uses BERTScore, RAGAS, a style classifier, BLEU, ROUGE-L, G-Eval and human evaluation. [filecite:turn0file2](#) [filecite:turn0file3](#) That is broad, but the design should be sharpened.

RAGAS was explicitly designed to separate retrieval quality, context use/faithfulness and generation quality in RAG systems. ¹⁰ BLEU and ROUGE originated as reference-overlap metrics for machine translation and

summarization, respectively; they can be informative secondary diagnostics but should not become the primary measure of open-ended persona writing. ¹¹ G-Eval was designed as an LLM-based evaluator and reported better human alignment on its studied NLG tasks, but its authors also note potential bias toward LLM-generated text; broader LLM-as-judge work identifies position, verbosity and self-enhancement biases.

¹²

Use this matrix instead:

Dimension	Metric	Applies to	Interpretation
Data integrity	source counts, retention %, duplicate rate, null rate	all	Pipeline correctness
Leakage	train↔val↔test duplicate/near-duplicate overlap	all	Must be zero at group level
Retrieval	Recall@k, MRR or nDCG on annotated query set	M3/M5	Did retrieval find the right evidence?
Grounding	RAGAS faithfulness + manual claim verification	M3/M5	Are claims supported by context?
Persona/style	stylometric distance; held-out authorship classifier AUC/F1	M1–M5	Does text resemble target distribution?
Semantic usefulness	structured human usefulness rating	M1–M5	Is output actually usable?
Reference similarity	BERTScore; ROUGE-L; BLEU as diagnostics	M1–M5	Similarity, not sufficient proof of persona quality
Format	deterministic YAML-spec validator	M1–M5	Word count/required sections/banned patterns
Voice quality	blinded 1–5 human voice rating	M1–M5	Human-perceived persona match
Factuality	blinded pass/fail or claim-level scoring	M1–M5	Strategic/numerical claim correctness
Operational	median/p95 latency, tokens/sec, peak VRAM	M1–M5	Practical system cost
Memorization	exact/near-copy rate or longest overlap	M2–M5	Detect imitation through copying rather than learned style

A recent authorship-style-transfer paper such as TinyStyler explicitly uses both automatic and human evaluation, reinforcing the need not to reduce style transfer to a single overlap metric. ¹³ Work on style-based RAG also supports treating authorial style and retrieved content as distinct evaluable dimensions.

¹⁴

Fair comparison protocol

For every model comparison:

- 1. Use the **same frozen evaluation prompts**.
- 2. Prevent test documents from entering persona-profile extraction, few-shot examples, retrieval indexes used for evaluation, or fine-tuning data unless the experiment explicitly defines otherwise.
- 3. Hold generation length and major decoding settings constant where meaningful.
- 4. Run multiple generations for stochastic settings rather than comparing one lucky output.
- 5. Store prompt-level results, not just aggregate means.
- 6. Report mean, spread and failures; do not report only the best candidate.
- 7. Use paired comparisons across the same prompts.
- 8. Keep the human-evaluation model identity blinded.
- 9. Preserve raw model outputs alongside metrics.
- 10. Record Git SHA, dataset hash, config hash, environment and hardware for every run.

Compute strategy under unknown resources

The project should be designed in tiers rather than assuming access to one GPU:

Compute available	Safe scope
CPU / ordinary laptop	Data QA, DuckDB, preprocessing, unit tests, small retrieval experiments, document generation with appropriately quantized local models at slower speed
~12-16 GB NVIDIA VRAM	Attempt 8B-class QLoRA only with conservative sequence length/batching and measured memory; run a small pilot first
24+ GB GPU	More comfortable fine-tuning/evaluation and larger batches
Temporary A100/ class GPU	Use for final M4/M5 training only after data/config freeze

The earlier slides report a 15.92 GB RTX 5070 Ti setup, while the prior workbook referred to a Colab A100, so the project has apparently contemplated both local and cloud training paths. [filecite turn0file2](#) [filecite turn0file3](#) QLoRA materially reduces memory relative to full fine-tuning, but its paper should not be interpreted as a guarantee that every 8B configuration fits every 16 GB card; sequence length, optimizer state, batch size and implementation still matter. 9

For the assignment, the right rule is: **a reproducible M1-M3 comparison is worth more than an unfinished M4 training job.**

Development workflow and agentic AI operating model

The fastest safe workflow is not “let one autonomous agent run the entire project.” It is a pipeline of narrowly scoped agents whose outputs are checked by deterministic tests and human gates.

Recommended engineering baseline

Move to a simple protected-main workflow:

```
main
├─ feat/data-manifest-v2
├─ feat/split-leakage-tests
├─ feat/m3-retrieval-eval
├─ exp/m4-qlora-v3
├─ exp/m5-hybrid-v1
└─ docs/demo1
```

Avoid another semester of long-lived architectural branches such as `scrapers`, `etlpipeline`, and `etl_jsonl`. Those made sense during exploration, but at this stage they encode different realities rather than independent features. The prior workbook itself documents substantial architectural divergence across them. [filecite](#) [turn0file3](#)

Every pull request should contain:

```
Purpose
Rubric criterion affected
Files changed
Data/config version
Tests added or updated
Commands run
Expected behavior
Experiment impact
Known limitations
Screenshots/metrics when applicable
```

For Python dependency reproducibility, a `pyproject.toml` plus committed `uv.lock` is a strong fit because `uv.lock` records exact resolved versions and is intended for consistent installations across machines. ¹⁵ Ruff can cover both formatting and linting from the same configuration, and GitHub Actions supports Python builds/tests with pytest. ¹⁶

A minimal CI gate should be:

```
uv lock --check
uv sync --locked
ruff format --check .
ruff check .
pytest -q
```

Do **not** run scraping, full embedding builds, Ollama downloads, or GPU training in routine CI. Test those interfaces with tiny fixtures and separate integration jobs.

Agentic-tool comparison

Tool	Best role here	Strength	Main caution	Recommendation
Cursor Ask / Plan	Repository ingestion, mapping, debugging plans, rubric analysis	Read-only exploration is available; Plan can precede edits	Can still reason from stale context if wrong branch opened	Primary discovery mode
Cursor Agent	Bounded implementation, tests, refactors	Multi-file exploration, edits and command execution	Broad prompts lead to broad diffs	Primary implementation agent
Cursor background/ cloud agents	Independent low-risk chores, documentation, isolated tests	Async remote execution	Internet access + automatic command execution; Cursor explicitly warns of prompt-injection/data-exfiltration risk	Use selectively, no credentials/ untrusted research ingestion
OpenAI Codex	Parallel issues/ reviews/alternative implementation	Current Codex supports parallel work, worktrees/ cloud environments and project workflows	Parallel work magnifies merge/config drift without tight tasks	Excellent optional second agent
GitHub Copilot review	PR review layer	Repo- and path-specific instructions can shape review	Not a substitute for tests or human experiment review	Use as supplemental reviewer
Custom multi-agent framework	Usually unnecessary here	Full programmable orchestration	Setup/ debugging consumes scarce assignment time	Do not add unless itself is a project requirement

Cursor documents Agent as capable of autonomous multi-file editing, while Ask is read-only. Its newer local execution modes add sandbox/review behavior; project rules can be committed under `.cursor/rules` and scoped to code areas. ¹⁷

Codex's current product supports parallel agent workflows and built-in worktrees; OpenAI also documents sandboxing that limits agents to their working folder/branch by default and seeks approval for elevated actions such as network access. ¹⁸ GitHub Copilot code review can consume repository instructions, path-specific instructions and `AGENTS.md`, making the same project standards reusable in PR review. ¹⁹

Repository agent instructions

Create both `AGENTS.md` and focused Cursor rules. `AGENTS.md` provides tool-agnostic project context, while Cursor project rules are version-controlled in `.cursor/rules`. ²⁰

A strong root instruction is:

Project invariants

Source of truth

- Current persona: Jason Lemkin / SaaStr.
- Canonical dataset and split versions are defined under data/manifests/.
- Canonical experiment configurations live under configs/.
- Never copy numerical results from old slides into current results.

Data

- Never modify raw data in place.
- Never move a record between splits after evaluation begins.
- Never use test records in persona building, training, prompt examples, or tuning.
- Every transformation must be deterministic or record a seed.

Experiments

- Do not change more than one major experimental variable unless explicitly requested.
- Every run must write git SHA, dataset version, config, seed, hardware, and metrics.
- Never label a model "validated" merely because code executes.

Coding

- Add/update tests with behavioral changes.
- Run Ruff and pytest before declaring completion.
- Do not commit secrets, large datasets, vector indexes, model weights, or credentials.

Reporting

- Distinguish Implemented, Tested, Evaluated, Proposed, and Planned.

- Never invent benchmark values.
- Every result used in slides must trace to a machine-readable results file.

Agent roles and orchestration

A high-efficiency orchestration pattern is:

Agent role	Permission	Task	Human checkpoint
Evidence agent	Read only	Rubric/research/prior-work crosswalk	Approve scope
Repo cartographer	Read only	Map modules, configs, DAGs, tests and drift	Confirm source-of-truth branch
Data-QA agent	Limited branch	Implement manifests/leakage/schema tests	Approve frozen dataset/splits
Implementation agent	Issue branch	One feature at a time	Review diff + CI
Experiment agent	Config + execution only	Execute frozen manifests	Approve run before GPU cost
Evaluation agent	Prefer read only	Compute metrics/tables from raw results	Audit metric code
Review agent	Read only	Search bugs, leakage, contradictions	Human merge decision
Documentation agent	Docs only	Populate workbook/slides from validated artifacts	Human claim sign-off

A useful implementation prompt is:

You are implementing exactly GitHub issue <ID>.

Before editing:

1. Read AGENTS.md, the relevant .cursor/rules, tests, and configuration.
2. State the files you expect to modify and acceptance tests.
3. Do not alter dataset splits, experiment targets, metrics, or unrelated architecture.

During implementation:

- Make the smallest coherent change.
- Add or update tests.
- Preserve backward compatibility unless issue explicitly says otherwise.
- Do not silently repair unrelated problems.

Before completion:

- Run the specified lint/test commands.
- Show the exact commands and results.
- Summarize changed files.
- List unresolved risks.
- Do not claim performance improvement unless an approved experiment demonstrates it.

An experiment-agent prompt should be even stricter:

Run experiment <experiment_id> from the committed configuration.

Do not:

- alter the frozen test set;
- tune based on test results;
- modify model/data configuration during the run;
- select only favorable outputs;
- overwrite previous raw results.

Record:

git SHA, dirty/clean status, dataset hash, split hash, model revision, all parameters, random seeds, GPU/CPU/RAM information, command, start/end timestamps, raw outputs, metrics and errors.

If configuration and implementation disagree, STOP and report the mismatch.

Human-in-the-loop gates

The human checkpoints should occur **before** irreversible or epistemically important actions, not after an agent has already done them:

Rubric gate → source-of-truth branch gate → dataset/split freeze → experiment protocol freeze → PR approval → results sign-off → slide/demo claim sign-off.

No agent should autonomously change split logic and then evaluate against the new split; change evaluation metrics after seeing results; merge its own experimental code; or convert an exploratory result into a “project result.”

Research PDFs and arbitrary web pages require additional caution. Cursor's background-agent documentation explicitly notes that internet-connected, auto-running background environments introduce prompt-injection/data-exfiltration risks. ²¹ Therefore: ingest documents through read-only tooling, never place secrets in agent context, use least-privilege credentials, and do not let instructions embedded in papers or scraped webpages override project rules.

Prioritization, timeline, and assignment deliverables

The project should optimize first for **grade coverage**, then scientific rigor, then additional architecture.

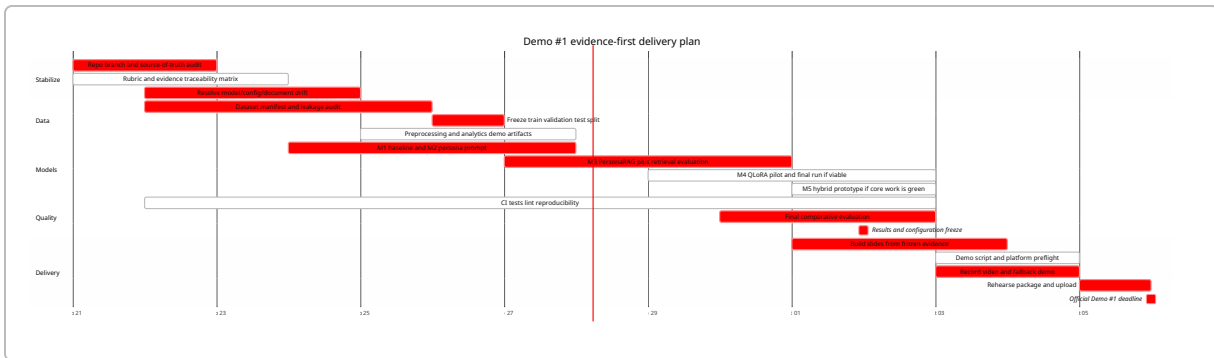
Priority stack

Priority	Work	Reason
P0	Normalize source-of-truth branch, manifests and model naming	Nothing else is trustworthy until project identity is stable
P0	Validate/rebuild current Jason dataset split and leakage tests	Evaluation integrity
P0	Re-run preprocessing and analytics artifacts from current data	Directly worth 5/12.5 demo points across preprocessing + analytics
P0	M1 and M2 reproducible baseline	Guarantees “ongoing ML results” even if retrieval/training fails
P1	M3 PersonaRAG + retrieval evaluation	Strong project differentiation with manageable compute
P1	CI, tests, linting, locked environment	Prevent deadline regression
P1	Build demo slide evidence automatically from current artifacts	Prevent slide/code mismatch
P2	M4 QLoRA verified run	High value only after P0/P1
P2	M5 hybrid experiment	Workbook proposal required; demo run optional
P3	LightRAG/graph extensions, extra orchestration, new infrastructure	Defer until core deliverables are complete

This ordering deliberately differs from a research-first roadmap. Demo #1 awards 2.5 points each to preprocessing, train/test preparation, analytics, ongoing ML and presentation, so a polished model that leaves data evidence weak is a poor optimization of the rubric. [filecite turn0file1](#)

Timeline to Demo #1

The Mermaid plan uses dates rather than times; **the actual deadline is October 6 at 12:00 PM**, so October 5 should be treated as the operational submission deadline. [filecite turn0file1](#)



The unknown workbook deadline should not produce a second speculative date. Instead maintain the workbook continuously as a **living report generated from the same evidence**, then schedule a T-3 day editorial freeze when its actual due date becomes available.

Workbook deliverable checklist

The workbook should be built directly around the grading rows rather than around an organically evolving project narrative. `filecite{turn0file0}`

Rubric evidence	Required deliverable
Model proposals	Canonical M1–M5 table with problem, concepts, inputs/features, algorithm, architecture, expected output, compute; identify M5 explicitly as improvement
Model supports	Per-model environment table; one system architecture diagram; one ML/data-flow diagram
Comparison/justification	Side-by-side approach/strength/limitation/latency/interpretability/grounding/training comparison
Evaluation methods	Per-model metric matrix and why each metric answers a specific hypothesis
System requirements	Explicit system boundary, actors, use cases, functional/non-functional requirements
System design	Data layer, retrieval layer, model layer, APIs, Streamlit/UI, orchestration, local environment and optional cloud-training lane
Intelligent solution	Inputs → outputs → model/service/API descriptions; status labeled Proposed / Implemented / Tested / Evaluated
Support environment	Reproducible version table, hardware, OS/CUDA/container assumptions, run instructions
Evidence integrity	Git SHA, dataset version/hash, experiment IDs and result artifact locations
Research rigor	Primary citations directly tied to design claims, not merely a long bibliography

The system-design rubric explicitly mentions integration with a cloud environment, while the previous project has generally emphasized local inference. [filecite turns 0 file 0](#) The clean solution is to describe a **local-first deployment with an optional cloud/hosted-GPU training lane**, not to pretend that inference depends on cloud services.

Demo deliverable checklist

Demo scoring area	Evidence that should appear on screen
Data preprocessing	Raw vs cleaned examples; eight operations; attrition funnel; source-by-source counts; dedup example
Training/test preparation	Current split counts; split proportions; seed; group/doc-level methodology; leakage check with zero overlaps
Data analytics	Source distributions; word lengths; stylometric distributions; persona vocabulary/themes; B2B keyword analysis
Ongoing ML	M1/M2/M3 outputs on identical prompts; quantitative table; retrieval evidence; M4 result only if genuinely run
Platform status	Short live route from input → selected model → generated output; optionally Airflow/data-layer status
Presentation	Clear problem → data → preprocessing → experiments → results → system → next step story
Submission	Final PPT/PPTX and recorded video

The old presentation is already visually sophisticated, so the objective should not be to redesign everything. Its greatest weakness is **semantic consistency**: the model names/configurations and claims need to match the final system. [filecite turns 0 file 2](#)

A safe five-minute failure-tolerant demo should also contain pre-rendered screenshots, one known working prompt, cached model/index artifacts, and a short recorded output in case Docker, Ollama, GPU drivers, or networking fail during presentation.

Risks, controls, and reusable templates

The project has enough moving parts that the dominant failure modes are now governance and reproducibility rather than basic coding.

Risk register

Risk	Probability	Impact	Mitigation
Default repo points to legacy Paul Graham implementation	High	Critical	Normalize/merge production branch immediately; record canonical SHA. ¹

Risk	Probability	Impact	Mitigation
Only four models against five-model “Excellent” workbook criterion	High	High	Add M5 hybrid proposal and explicitly distinguish proposal from implementation. filecite turn0file0
Historical train/test manifest belongs to old corpus	High	Critical	Rebuild split from current source manifests; group duplicates before split; automated leakage test. filecite turn0file3
Model/configuration drift across workbook, slides and code	High	High	Central project/model config; generate documentation/results from it. filecite turn0file2 filecite turn0file3
Claims such as “built” or “evaluation ready” exceed actual validated state	Medium-high	High	Status taxonomy: Proposed → Implemented → Smoke-tested → Evaluated → Validated; every number links to raw result.
Fine-tuning consumes remaining schedule	Medium	High	Pilot with small sample first; Oct 2 hard stop; M1–M3 remain sufficient demo backbone.
Dependency/environment drift	Medium-high	High	<code>pyproject.toml</code> , committed <code>uv.lock</code> , pinned containers/model revisions, reproducibility command. ²²
No deterministic quality gate before agent merges	Medium-high	High	Ruff + pytest + integration fixtures + protected main. ²³
Agent prompt injection/ data exfiltration	Medium	High	Research ingestion read-only; no secrets; sandbox/least privilege; avoid background-agent auto-run on untrusted content. ²⁴
Persona imitation is confused with endorsement/authentic speech	Medium	High	Demo/UI disclosure that output is a style simulation; provenance and factual-grounding checks; do not present generated claims as actual quotations
Memorization/copying masquerades as high style fidelity	Medium	High	Near-copy metric; held-out evaluation; inspect longest phrase matches
Live demo infrastructure failure	Medium	High	Preflight script, frozen local bundle, cached outputs, backup video/screenshots
Research corpus contaminates persona corpus	Low-medium	High	Separate stores/indexes and explicit collection names
Hand-entered slide metrics become stale	High	Medium	Generate tables/charts from <code>results.json</code> ; never manually retype final values

Recommended project template

A compact target repository layout is:

```
.
├── AGENTS.md
├── README.md
├── pyproject.toml
├── uv.lock
├── dvc.yaml
├── configs/
│   ├── project.yaml
│   ├── data.yaml
│   ├── models/
│   └── experiments/
├── src/persona_pipeline/
│   ├── collection/
│   ├── preprocessing/
│   ├── storage/
│   ├── retrieval/
│   ├── models/
│   ├── evaluation/
│   └── app/
├── dags/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fixtures/
├── data/manifests/
├── experiments/
├── notebooks/
├── research/
├── reports/
├── docs/
│   ├── rubric_matrix.md
│   ├── architecture.md
│   ├── codebase_map.md
│   └── project_inconsistencies.md
├── .cursor/rules/
└── .github/workflows/
```

README template

AI-Driven Persona Cloning for B2B Content Generation

Project question

What are we predicting/generating, for whom, and why?

Assignment traceability

- Workbook rubric matrix: docs/rubric_matrix.md
- Demo artifacts: reports/demo1/

Current status

Model	Proposed	Implemented	Tested	Evaluated	
...	...	...	...	...	

Architecture

Canonical architecture diagram and data flow.

Data

- sources
- provenance
- dataset version
- schema
- split methodology
- leakage controls

Models

M1–M5 with one paragraph each.

Reproducibility

Exact setup commands.

Tests

Exact lint/test commands.

Experiments

How to execute a committed experiment config.

Results

Only frozen, reproducible results.

Demo

How to start local services and run a known-good example.

Ethics and limitations

Persona simulation disclosure, provenance, copying/hallucination limitations.

Contributors

Roles and ownership.

Experiment-log template

Every experiment should write a machine-readable record such as:

```
experiment_id: m3_personarag_v4
timestamp_utc: 2026-09-29T03:20:00Z
```

```
provenance:
  git_sha: "...
  git_dirty: false
  dataset_version: lemkin-v2
  dataset_hash: "...
  split_hash: "...
  prompt_set_version: demo-eval-v2
```

```
environment:
  os: "...
  python: "...
  uv_lock_hash: "...
  gpu: "...
  driver: "...
  cuda: "...
```

```
model:
  name: m3_personarag
  base_model: "...
  base_model_revision: "...
  generation_config:
    temperature: 0.7
    top_p: 0.9
```

```
retrieval:
  embedding_model: "...
  chunk_size: 500
  overlap: 50
  top_k: 5
  reranker: "...
```

```
randomness:
  seed: 42
```

```
execution:
  command: "...
  started_at: "...
  ended_at: "...
```

```
metrics:
```

```
retrieval_recall_at_5: null
faithfulness: null
style_metric: null
format_compliance: null
latency_ms: null

artifacts:
  raw_outputs: "...
  retrieved_contexts: "...
  metrics_json: "...
  figure_dir: "...

notes: ""
```

A run with `null` metrics is incomplete, not failed and certainly not “validated.” That distinction prevents agents or slide authors from turning execution status into scientific conclusions.

Notebook template

Notebooks should be presentation/analysis interfaces, not hidden pipelines:

```
Title and research question
Input artifact versions/hashes
Environment and seed
Data-quality checks
Analysis or experiment
Figures/tables
Result
Interpretation
Failure cases
Limitations
Artifact export
```

The standard for a notebook should be **restart kernel → run all → same outputs from the declared inputs**. Core preprocessing, splitting and metric logic should live in importable/tested Python modules rather than notebook cells.

Minimal definition of done

A task is only complete when:

```
Code merged from a reviewed branch
+ tests pass
+ formatting/lint passes
+ data/config version identified
```

- + result artifact preserved if applicable
- + rubric evidence updated
- + documentation matches code
- + no fabricated or manually transcribed result

That definition is more important than which coding agent is used. Cursor, Codex and Copilot can all accelerate different parts of the workflow, but the decisive quality improvement comes from giving them **stable project context, narrow tasks, deterministic acceptance tests, versioned evidence, and explicit human gates**. Cursor's project rules are specifically designed to encode persistent repository guidance, while GitHub Copilot review can consume repository and path-specific instructions; Codex provides isolated workspace-style parallelism when genuinely independent tasks need to run simultaneously. ²⁵

For this project, the optimal outcome by October 6 is therefore not “the largest possible agentic system.” It is a **single canonical repository, a frozen and leakage-checked current dataset, reproducible M1–M3 results with M4/M5 added only when stable, automated quality gates, and a workbook/demo whose every architecture claim and numerical result traces back to a versioned artifact**. That approach directly matches the provided rubrics while also producing a codebase that can continue into the subsequent project phase without another round of architectural reconciliation.

¹ <https://github.com/andreahcruz/298P>

<https://github.com/andreahcruz/298P>

² ¹⁵ ²² <https://docs.astral.sh/uv/concepts/projects/layout/>

<https://docs.astral.sh/uv/concepts/projects/layout/>

³ ¹⁷ <https://docs.cursor.com/agent>

<https://docs.cursor.com/agent>

⁴ <https://developers.google.com/workspace/drive/api/reference/rest/v3>

<https://developers.google.com/workspace/drive/api/reference/rest/v3>

⁵ <https://prod.cursor.com/help/ai-features/agent>

<https://prod.cursor.com/help/ai-features/agent>

⁶ <https://dvc.org/doc/command-reference/>

<https://dvc.org/doc/command-reference/>

⁷ <https://arxiv.org/abs/2005.11401>

<https://arxiv.org/abs/2005.11401>

⁸ <https://arxiv.org/abs/2407.09394>

<https://arxiv.org/abs/2407.09394>

⁹ <https://arxiv.org/abs/2305.14314>

<https://arxiv.org/abs/2305.14314>

¹⁰ <https://arxiv.org/abs/2309.15217>

<https://arxiv.org/abs/2309.15217>

¹¹ <https://aclanthology.org/P02-1040/>

<https://aclanthology.org/P02-1040/>

12 <https://arxiv.org/abs/2303.16634>

<https://arxiv.org/abs/2303.16634>

13 <https://aclanthology.org/2024.findings-emnlp.781/>

<https://aclanthology.org/2024.findings-emnlp.781/>

14 <https://aclanthology.org/2024.personalize-1.11/>

<https://aclanthology.org/2024.personalize-1.11/>

16 23 <https://docs.astral.sh/ruff/>

<https://docs.astral.sh/ruff/>

18 <https://openai.com/codex/>

<https://openai.com/codex/>

19 <https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review>

<https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review>

20 25 <https://docs.cursor.com/context/rules>

<https://docs.cursor.com/context/rules>

21 24 <https://docs.cursor.com/background-agent>

<https://docs.cursor.com/background-agent>